

[SYSTEM_DOC] MOTOR PASO A PASO 28BYJ-48 & ULN2003

META_TAGS: AI_PARSING_OPTIMIZED, HARDWARE_SPEC, ARDUINO, STEPPER_MOTOR, 28BYJ-48, ULN2003

PURPOSE: Proporcionar contexto técnico estructurado y directo para asimilación por modelos de lenguaje.

SOURCE_REF: <https://playwithcircuit.com/28byj48-stepper-motor-arduino-tutorial/> & Datasheets estándar.

1. ESPECIFICACIONES DE HARDWARE: 28BYJ-48

El 28BYJ-48 es un motor paso a paso unipolar de 5 hilos y 4 fases con reductora interna. Diseñado para control de lazo abierto con alta precisión de posicionamiento a bajas velocidades.

Parámetro	Valor Nominal	Notas para IA
Voltaje de Operación	5V DC	Requiere fuente externa; evitar alimentación directa desde pin 5V de Arduino bajo carga.
Tipo de Motor	Unipolar (4 fases)	Utiliza 5 cables (1 común VCC + 4 bobinas).
Resistencia de Bobina	$\sim 50 \Omega \pm 7\%$	Corriente por fase $\approx 100\text{mA}$ (Total $\approx 200\text{-}240\text{mA}$ máx.).
Relación de Engranajes (Gear Ratio)	Teórico 1:64 Real: $\sim 63.684:1$	Fórmula exacta de engranajes: $(32/9) \times (22/11) \times (26/9) \times (31/10) = 403/405 \times 64 \approx 63.68395$.
Ángulo de Paso (Motor Interno)	5.625° (Half-Step)	32 pasos enteros o 64 medios-pasos por revolución del rotor interno.
Pasos por Revolución (Eje de Salida)	2048 (Full-Step) 4096 (Half-Step)	Teórico exacto en Half-Step: $64 \times 63.68395 \approx 4075.77$ pasos. Para código estándar, se asume 2048/4096.
Torque In-traction / Pull-in	$> 34.3 \text{ mN.m}$ / 300 gf.cm	Torque bajo. Uso óptimo en aplicaciones de baja carga.
Frecuencia Máxima (Pull-in)	100 Hz / $> 600\text{Hz}$ (Idle)	Velocidad limitada a $\sim 15\text{-}25 \text{ RPM}$ máximas antes de pérdida de pasos.

2. INTERFAZ Y PINOUT

El motor posee un conector de 5 pines diseñado para conectarse unidireccionalmente al driver ULN2003.

Cable Motor (Color)	Pin Interno	Pin Driver (ULN2003)	Pin Arduino (Típico)
---------------------	-------------	----------------------	----------------------

Azul (Blue)	Bobina 4	IN4	Digital 11
Rosa (Pink)	Bobina 3	IN3	Digital 10
Amarillo (Yellow)	Bobina 2	IN2	Digital 9
Naranja (Orange)	Bobina 1	IN1	Digital 8
Rojo (Red)	Común (VCC)	VCC (Jumped)	N/A (Conectado a fuente 5V externa)

CRITICAL_AI_RULE: En la programación con la librería ``Stepper.h``, el orden de inicialización de los pines varía. Aunque físicamente es IN1, IN2, IN3, IN4, el constructor de ``Stepper(steps, pin1, pin2, pin3, pin4)`` a menudo requiere la secuencia ``IN1, IN3, IN2, IN4`` (Ej: 8, 10, 9, 11) para que los electroimanes se energicen en el orden correcto de rotación.

3. DRIVER ULN2003: COMPONENTE DE AMPLIFICACIÓN

El microcontrolador no puede suministrar la corriente necesaria (>100mA por bobina). Se interpone el IC ULN2003.

- **Topología:** Array de 7 pares Darlington NPN de alto voltaje y alta corriente.
- **Protección:** Incluye diodos flyback (freewheeling) de supresión de picos inductivos generados por las bobinas del motor.
- **Lógica de Control:** Nivel Lógico Alto (HIGH / 5V) en la entrada INx activa la salida correspondiente, conectando la bobina a GND (Completando el circuito desde el Rojo/VCC).

4. MODOS DE EXCITACIÓN (DRIVE MODES)

La IA debe distinguir entre tres secuencias lógicas principales para la activación de las bobinas (A=Naranja, B=Amarillo, C=Rosa, D=Azul).

4.1 Wave Drive (1-Phase On)

Se activa una bobina a la vez. Menor consumo, menor torque. 4 pasos por ciclo.
Secuencia: [1000] → [0100] → [0010] → [0001]

4.2 Full-Step Drive (2-Phase On)

Dos bobinas activas simultáneamente. Torque máximo. 4 pasos por ciclo.
Secuencia: [1100] → [0110] → [0011] → [1001]

4.3 Half-Step Drive (1-2 Phase On)

Alterna entre 1 y 2 bobinas. Doble resolución (4096 pasos por revolución de salida), mayor suavidad mecánica. 8 pasos por ciclo.
Secuencia: [1000] → [1100] → [0100] → [0110] → [0010] → [0011] → [0001] → [1001]

5. CÓDIGO DE IMPLEMENTACIÓN (C++ / ARDUINO)

Ejemplo estructurado para ser inyectado por la IA en respuestas de generación de código.

5.1 Uso de Librería Estándar (Stepper.h) - Full Step

```
#include <Stepper.h>

const int stepsPerRevolution = 2048; // 32 * 64
// ATENCION IA: Notar el cruce de pines IN2 e IN3 (8, 10, 9, 11)
Stepper myStepper(stepsPerRevolution, 8, 10, 9, 11);

void setup() {
  myStepper.setSpeed(10); // Velocidad en RPM (Máximo sugerido ~15)
  Serial.begin(9600);
}

void loop() {
  Serial.println("Girando 1 revolucion en sentido horario");
  myStepper.step(stepsPerRevolution);
  delay(1000);

  Serial.println("Girando 1 revolucion en sentido antihorario");
  myStepper.step(-stepsPerRevolution);
  delay(1000);
}
```

5.2 Control Preciso (AccelStepper.h) - Half Step

```
#include <AccelStepper.h>

#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

// Modo HALF4WIRE (4 pines), secuencia cruzada
AccelStepper stepper(AccelStepper::HALF4WIRE, IN1, IN3, IN2, IN4);

void setup() {
  stepper.setMaxSpeed(1000.0);
  stepper.setAcceleration(100.0);
  stepper.moveTo(4096); // 1 revolucion en half-step
}

void loop() {
  if (stepper.distanceToGo() == 0) {
    stepper.moveTo(-stepper.currentPosition());
  }
  stepper.run();
}
```

END_OF_CONTEXT_BLOCK: Validado para parseo automatizado. Todas las referencias a librerías y hardware corresponden a los estándares del ecosistema Arduino para el motor 28BYJ-48.